

Python Graph Gallery

A part of the Reproducible Research Workshop Series

www.dartgo.org/RRADworkshops

Steve Gaughan

<https://dartgo.org/steve>

Research Computing & Data @ ITC

rc.dartmouth.edu





About the Reproducible Research Group

- Joint venture of **Research Computing @ ITC** and **Research Data Services @ Library**
- Consult with us on
 - Research data management
 - Data visualization
 - Biomedical research support
 - Spatial data and GIS
 - High performance and research computing
 - Statistical software and tools
 - Economics and social sciences data
- **Meet** the people on campus that support your reproducible research lifecycle
- **Engage** in community discussions to learn from other researchers on campus
- **Attend** our workshops to **learn** practical tools and tips



About Research Computing and Data

High Performance Computing

Linux Cluster and High
Performance Network Storage

Secure systems for medical and
protected data

Data Science, Data Analysis, Data Visualization

Textual, numerical, spatial data

Assistance with building
reproducible workflows

Scripting in R and Python

Consulting

Computational project planning

Storytelling with data and
visualizations

Digital Humanities support



A roadmap for this workshop

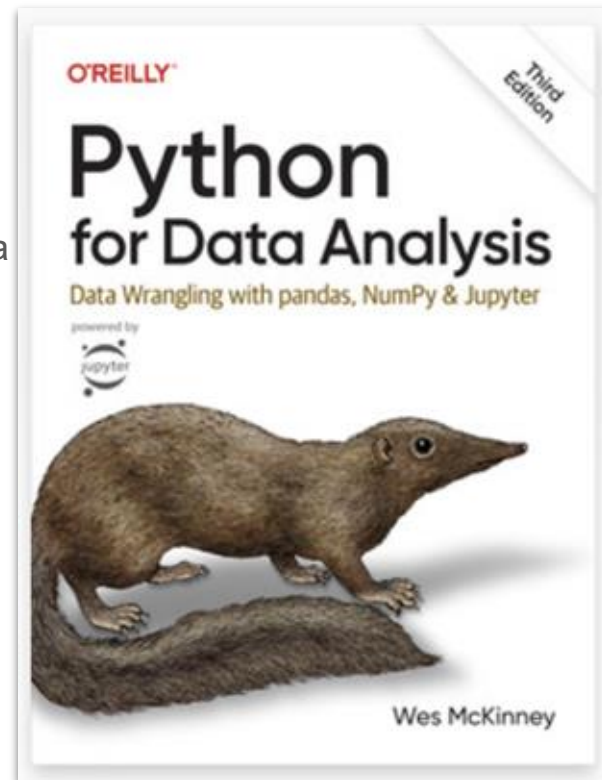
- Why Python for Graphs & Plots?
- Python Jupyter Notebooks
- [Tidy](#) Data
- Live-coding using Python with a reproducible plotting and graphing techniques
- Feel free to contact us anytime! [research.computing -at- dartmouth.edu](mailto:research.computing-at-dartmouth.edu)
- Research Computing homepage <https://rc.dartmouth.edu/>
- ITC homepage <https://services.dartmouth.edu/>



Why Python?

Python can help us make our research reproducible

- Python is open source, with many libraries built specifically for data science
- Python is a free and easy to download
- Standard libraries like Matplotlib and Seaborn for professional quality graphs and plots
- Python's importable libraries make it a useful language for data science and data analysis
- Python can import and export common data formats such as CSV files
- References: [Python Graph Gallery](#), [Python for Data Analysis](#), by Wes McKinney, O'Reilly Publishing
https://www.w3schools.com/datascience/ds_python.asp





Python Graph Gallery and Python for Data Science

https://python-graph-gallery.com/

← Graph Gallery CHART TYPES TOOLS ▾ ALL BEST LIBS RELATED ▾ LEARN

The biggest list of python chart examples

Within our collection, we cover **every chart type** imaginable to ensure we fulfill your visualization needs. To streamline the process of finding your required chart, we **classified** all the examples under their respective chart types.

For each chart type, we kick off with a **foundational tutorial** that introduces its basic and utility. After mastering the basics, users can delve into our step-by-step guides, elementary **customizations**, ensuring your chart not only presents data but does so with an individual touch that caters to your specific requirements.

Distribution



Violin Density Histogram Boxplot Ridgeline Beeswarm

Correlation



Scatterplot Heatmap Correlogram Bubble Connected Scatter 2D Density

w3schools.com/datascience/ds_python.asp

Tutorials ▾ References ▾ Exercises ▾ Certificates ▾ Search... Get Certif

AI R GO KOTLIN SWIFT SASS VUE GEN AI SCIPY AWS CYBERSECURITY

Python

Python is a programming language widely used by Data Scientists.

Python has in-built mathematical libraries and functions, making it easier to calculate mathematical problems and to perform data analysis.

We will provide practical examples using Python.

To learn more about Python, please visit our [Python Tutorial](#).

Python Libraries

Python has libraries with large collections of mathematical functions and analytical tools.

In this tutorial, we will use the following libraries:

- **Pandas** - This library is used for structured data operations, like import CSV files, create dataframes, and data preparation
- **Numpy** - This is a mathematical library. Has a powerful N-dimensional array object, linear algebra, Fourier transform, etc.
- **Matplotlib** - This library is used for visualization of data.
- **SciPy** - This library has linear algebra modules

We will use these libraries throughout the tutorial to create examples.

DS HOME
DS Introduction
DS What is Data
DS Database Table
DS Python
DS DataFrame
DS Functions
DS Data Preparation
DS Math
DS Linear Functions
DS Plotting Functions
DS Slope and Intercept
DS Statistics
Stat Introduction
Stat Percentiles
Stat Standard Deviation
Stat Variance
Stat Correlation
Stat Correlation Matrix
Stat Correlation vs Causality
DS Advanced
DS Linear Regression



Python Graphs & Plots - Libraries

Python can help us make our research reproducible with libraries that extend capabilities

- Some data science libraries include:
 - *NumPy* & *SciPy* - For numerical & scientific computations and linear algebra
 - *Pandas* (data frames) - Handling and manipulating data structures / data frames
 - *Matplotlib* - Data visualization
 - *Seaborn* - Statistical graphics, which build on matplotlib's foundation
 - *Plotly* - Interactive and dynamic data visualizations
 - *Scipy Stats and Statsmodels* - Econometric and statistical modeling
- Note: R & RStudio are a great option also, many similarities (data frames, ggplot, stats)
- More books and resources:
 - *Python for Data Analysis*, Wes McKinney, O'Reilly Publishing
 - *Data Science from Scratch: First Principles with Python 2nd Edition*, Joel Grus
 - *Python for Data Science: A Hands-On Introduction*, Yuli Vasiliev



Python Graphs & Plots - Libraries

Python can help us make our research reproducible with libraries that extend capabilities

- Some data science libraries include:
 - *NumPy* & *SciPy* - For numerical & scientific computations and linear algebra
 - Available through Dartmouth College Library
 - Online edition of “Foundational Python for Data Science”
<https://learning.oreilly.com/library/view/foundational-python-for/9780136624417/>
 - Python Graph Gallery <https://python-graph-gallery.com/>
 - Matplotlib <https://matplotlib.org/3.5.3/gallery/index.html>
 - Seaborn <https://seaborn.pydata.org/>
 - Python for Data Science https://www.w3schools.com/datascience/ds_python.asp



Matplotlib library for Data Visualization

[Matplotlib](#) pyplot



Plot types Examples Tutorials Reference User guide Develop Release notes

Lines, bars and markers

Images, contours and fields

Subplots, axes and figures

Statistics

Pie and polar charts

Text, labels and annotations

pyplot

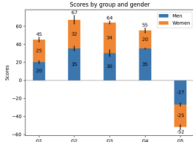
Color

Shapes and collections

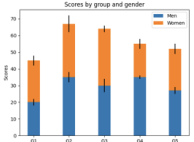
Style sheets

axes_grid1

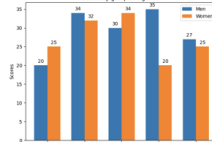
Lines, bars and markers



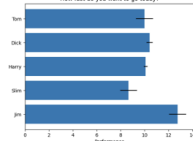
Bar Label Demo



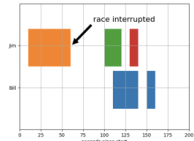
Stacked bar chart



Grouped bar chart with labels



Horizontal bar chart



Broken Barh

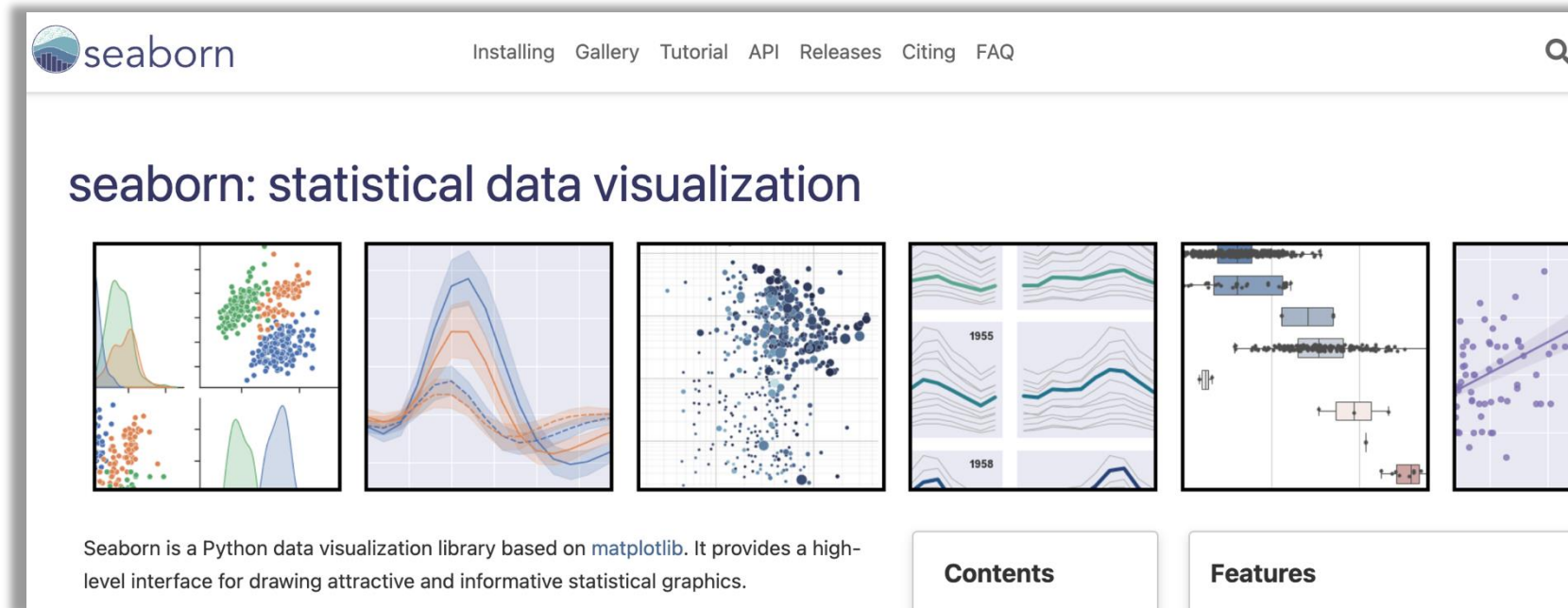


CapStyle

Seaborn library for Data Visualization

Seaborn

- Seaborn



The screenshot shows the Seaborn website homepage. At the top, there is a navigation bar with links: Installing, Gallery, Tutorial, API, Releases, Citing, and FAQ. The main heading is "seaborn: statistical data visualization". Below the heading is a grid of six thumbnail images showcasing various statistical plots created with Seaborn, including density plots, scatter plots, line plots with confidence intervals, and box plots. At the bottom of the page, there are two buttons: "Contents" and "Features".

seaborn: statistical data visualization

Seaborn is a Python data visualization library based on [matplotlib](#). It provides a high-level interface for drawing attractive and informative statistical graphics.

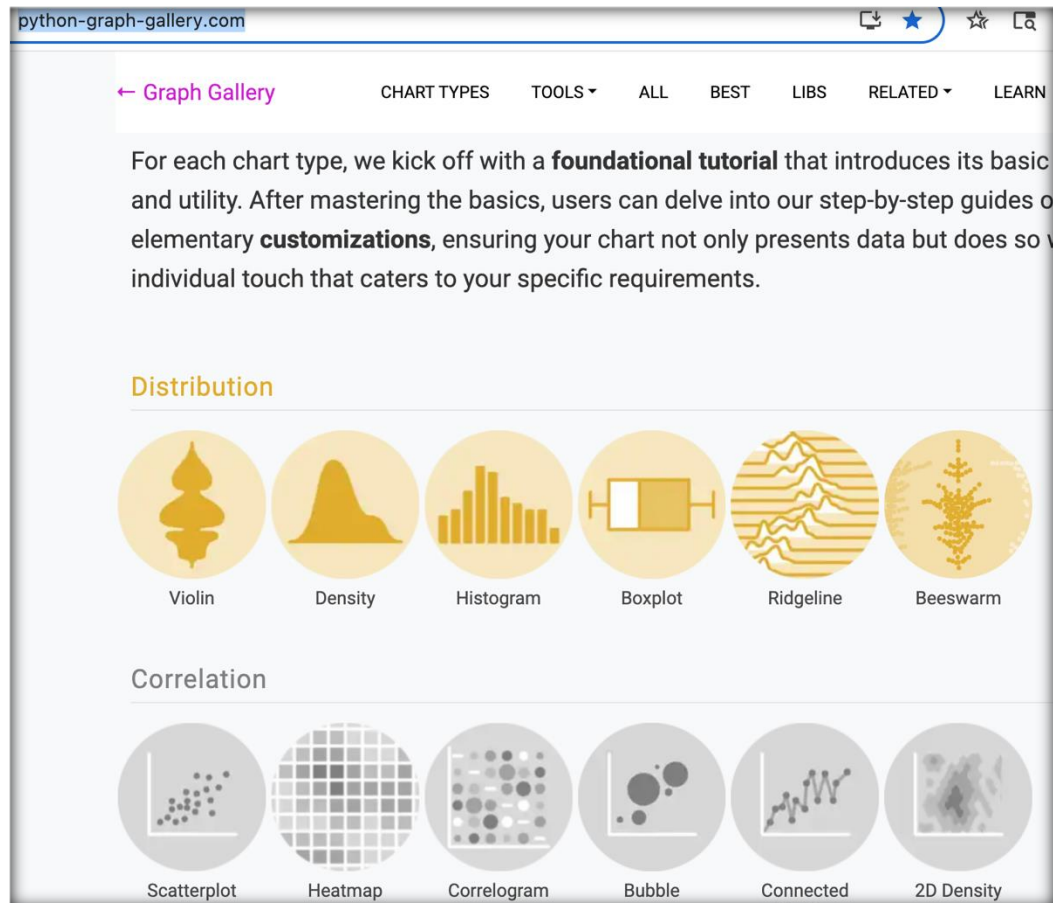
Contents **Features**



Python Graph Gallery

Python [Graph Gallery site](https://python-graph-gallery.com)

- Graph Gallery

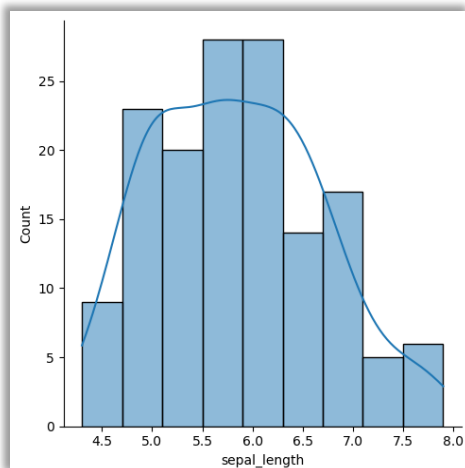




Live code with built in datasets

```
# library & dataset  
import seaborn as sns  
df = sns.load_dataset('iris')
```

```
# Plot the histogram thanks to the displot function  
sns.displot( data=df["sepal_length"], kde=True )
```





Basics of Reproducible Research for Data Science

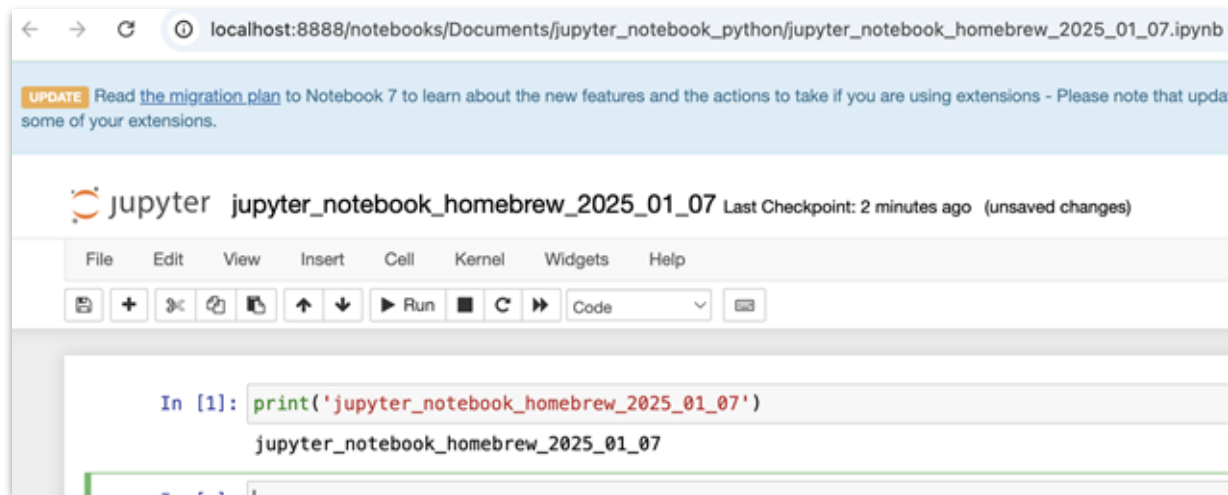
To make our research reproducible:

- Provide data, with metadata, and the code and software or software version used to run the analysis
- Emphasize transparency throughout the research lifecycle
- Test that you can generate the same result more than once
- Enable other researchers to be able to generate the same result, given the data and the scripts or programs that capture the methodology of the analysis, and then build on that with new data
- Run the same analysis steps, given new data with an identical data structure. For example, the data is updated with new observations(rows) and the analysis is re-run.



What is a Jupyter Notebook?

- Jupyter Notebooks store code, and also show things like plots, visualizations, maps and explanations generated by the Python code and 'markdown' text
- Types:
 - Anaconda Jupyter notebook
 - VS Code
 - Google Colab





Jupyter Notebooks, Anaconda, VS Code, Google Colab...

- Python = programming language
- [Anaconda](#) = downloadable software for Mac, PC, Linux that contains a Python Jupyter Notebook system within it
- [VS Code](#) = downloadable software for Mac, PC, Linux that can (with the right plugins) be used as a Python Jupyter notebook
- [Google Colab](#) = browser-based Google product (must have a Google account of some sort) that mimics a Python Jupyter notebook
- Jhub = "JupyterHub" and is a web or server-based Jupyter Notebook. Jhub.Dartmouth.edu jhub.dartmouth.edu is TEACHING & LEARNING platform, not designed for research purposes



Notebooks & Code for Python

Anaconda Navigator or Visual Studio Code with Python can help us make our research reproducible

- See these notebooks for more: <https://github.com/Dartmouth-Libraries/Intro-to-Python> and <https://github.com/Dartmouth-Libraries/data-viz-matplotlib>
- Anaconda navigator: <https://www.anaconda.com/download/success>
- Download and install VS Code(Mac or PC) <https://code.visualstudio.com/download>
- See <https://code.visualstudio.com/docs/python/python-tutorial>
- Set up Python in VS Code (Getting Started with Python in VS Code): <https://www.youtube.com/watch?v=D2cwvpJSBX4>
- More Youtube videos on the Visual Studio Code youtube channel at <https://www.youtube.com/@code>



Conda, Pip, Homebrew

- What is does “pip install seaborn” mean?
- Conda environments, [Anaconda website](#) environments, [Conda environments on Dartmouth systems](#) and , [Conda website](#)
- Pip is a [Python package installer](#) system
- Homebrew is a [Mac-only installer](#) system [Homebrew and Python](#)



File extensions: .py, .R, .ipynb, csv, xls, jpg, png

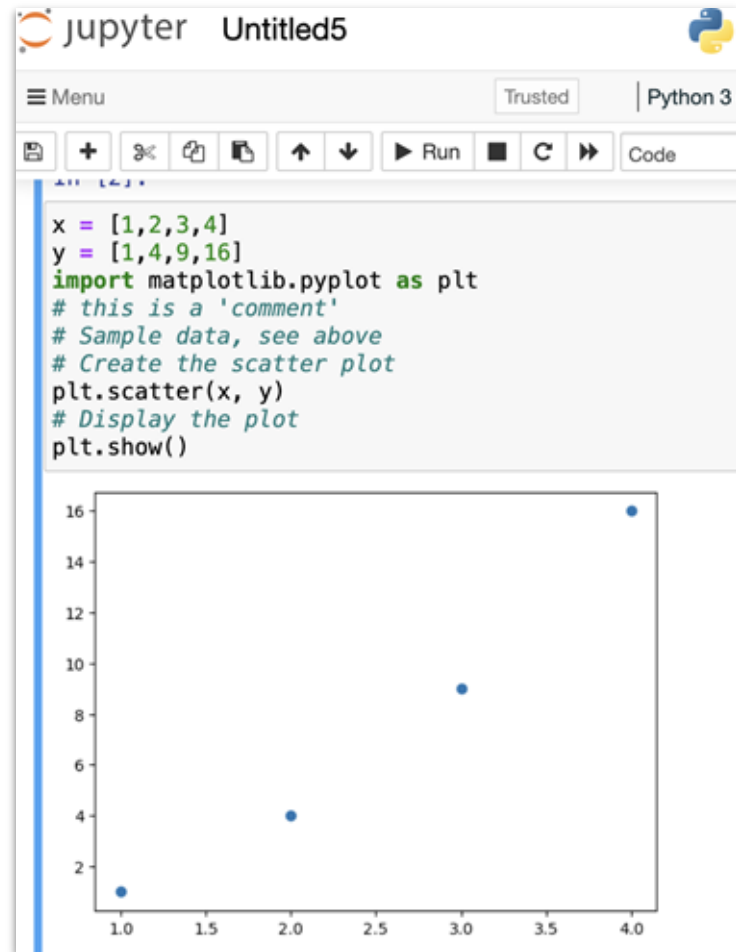
- Code (.py, .R)
- Notebook (code + text(markdown) + images) (.ipynb)
- Data (.csv, .xls)
- Tidy data – rows and columns, one row per observation, one data type per column, column headings with no spaces no special characters
- Images (.jpg, .png, .svg)
- Code + data + jupyter md = results, stats, plots

1	Adelie	Torge...	39.1	18.7	181
2	Adelie	Torge...	39.5	17.4	186
3	Adelie	Torge...	40.3	18	195
4	Adelie	Torge...	NA	NA	NA
5	Adelie	Torge...	36.7	19.3	193
6	Adelie	Torge...	39.3	20.6	190



With just a few lines of code, here's a very basic example of how Python and Jupyter notebooks can be used to create reproducible code and a visualization, both of which are essential to data science:

```
x = [1,2,3,4]
y = [1,4,9,16]
import matplotlib.pyplot as plt
# this is a 'comment'
# Sample datasets x and y, see above
# Create the scatter plot
plt.scatter(x, y)
# Display the plot
plt.show()
```



More code to run:

```
import seaborn as sns

# biology/ornithology/zoology example
penguins = sns.load_dataset('penguins')
# first five rows of data
penguins.head(5)

# shift run to run and create new cell
# built-in dataset - penguins
```

	species	island	bill_length_mm	bill_depth_mm
0	Adelie	Torgersen	39.1	18.7
1	Adelie	Torgersen	39.5	17.4



Source: <https://www.kaggle.com/code/sunaysawant/penguin-in-antarctica-eda>



Import seaborn as sns

```
import seaborn as sns  
penguins = sns.load_dataset('penguins')
```





Loading external data from Excel or CSV, and exporting data to CSV or Excel

```
import pandas as pd
# pandas is an acronym for python data
analysis / panel data attributed to Wes
McKinney

# Export the dataframe to a CSV file
penguins.to_csv('penguins.csv')

# Import the 'penguins' dataset from a csv
file into a new pandas 'data frame'
penguins_data_frame =
pd.read_csv('penguins.csv')
```

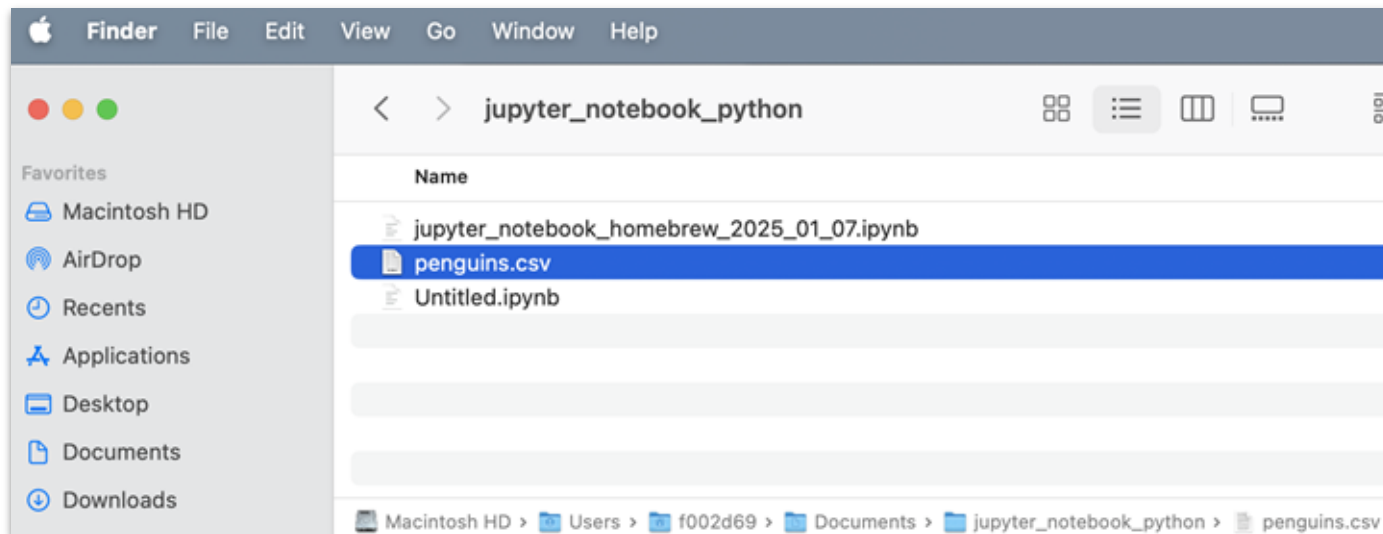
Or you can download the penguin data and then see if you can import it:

https://dartgo.org/penguin_data_csv

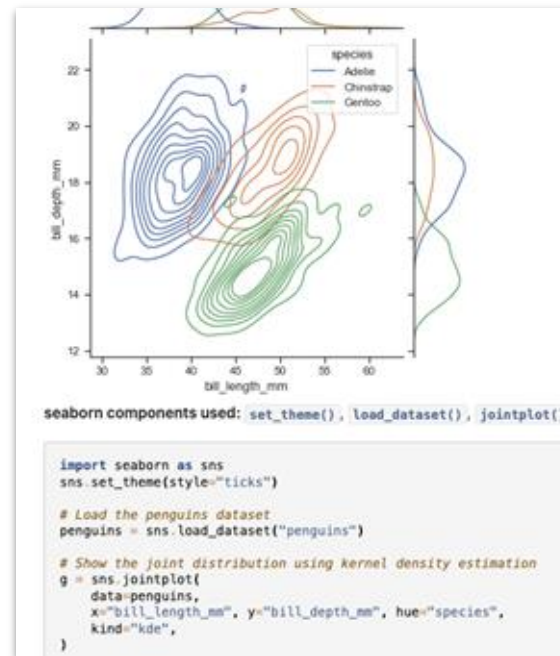
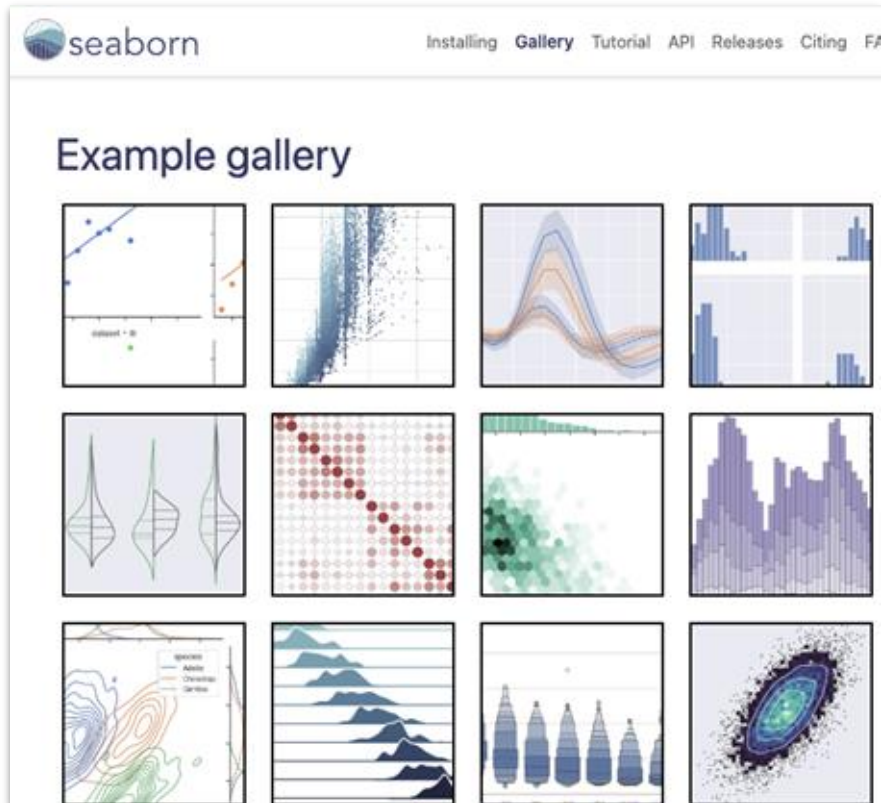


Export to CSV / Excel, import to CSV / Excel:

The Jupyter notebook, in this case, was in the user's "Documents" folder, so the exported CSV file was placed there



Seaborn plots...and code!



<https://seaborn.pydata.org/examples/index.html>



'Tidy' Data - a concept for all data, borrowed from R

Tidy Data refers to a structure for arranging data where:

- Each variable has its own column, making it easier to compare across observations.
- Each observation (or case) has its own row, allowing for easy handling and aggregation of the data.
- There is one table (not multiple tables) per subject area or concept.
- There are no hidden values, and the number of rows is equal to the total count of observational units.

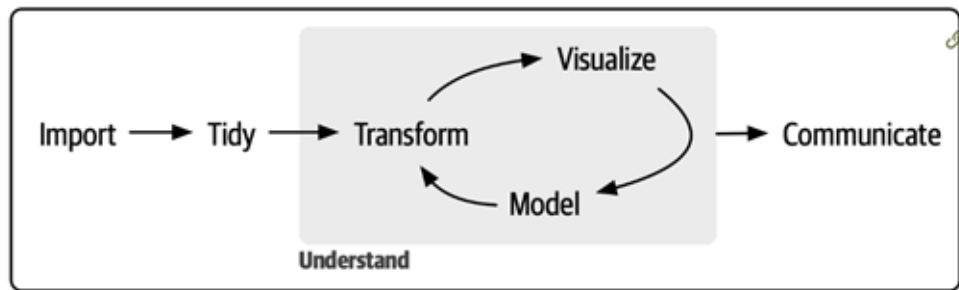
Following these principles allows for much easier data analysis, and it promotes an intuitive understanding and exploration of the data.

Note: it is ok to have multiple tables and relational database tables, but for data analysis and visualization, it is beneficial to focus on extracting data from those entities into a tidy-format data structure as described above. (source:

<https://r4ds.had.co.nz/tidy-data.html>

A typical data science process:

- **Good practice: make a copy of raw dataset, store securely if necessary, do not modify the raw data**
- With the copy, import & tidy or tidy & import
- Transform/model/analyze/visualize
- Review results with colleagues, iterate process for new methods & discovery
- Communicate the results to general audience in a visual, human-readable and understandable fashion



Program



Tidy data example, observations

- Observations per penguin
- Rows are individuals, columns are attributes/measurements, species, location where penguin was measured, length of the bill length, flipper length, body mass

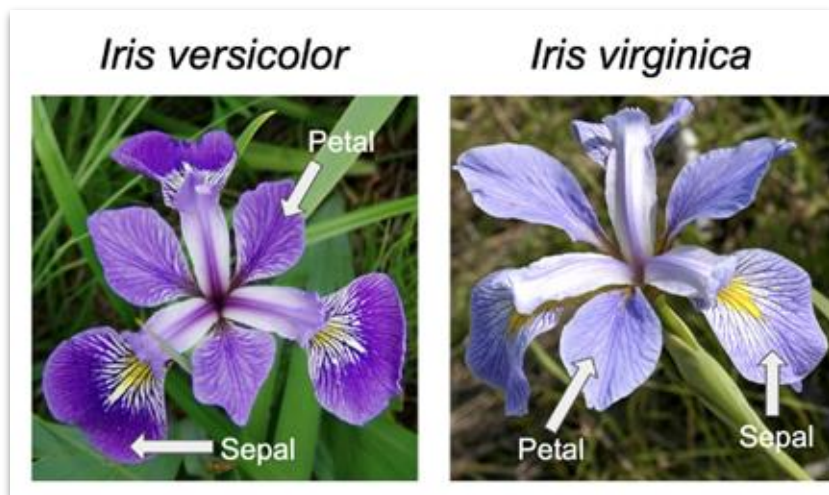
```
# first five rows of data  
penguins.head(5)  
# penguins.tail(5)
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0

Built-in datasets

Python's Seaborn library has very useful built-in datasets, specifically for learning data science and data visualization concepts

- 'Iris' dataset
- Penguins dataset



Sources: https://en.wikipedia.org/wiki/Iris_flower_data_set#/media/File:Iris_virginica.jpg
https://en.wikipedia.org/wiki/Iris_flower_data_set#/media/File:Iris_versicolor_3.jpg



Source: <https://www.kaggle.com/code/sunaysawant/penguin-in-antarctica-eda>



More resources

- Pandas
 - https://pandas.pydata.org/docs/getting_started/intro_tutorials/index.html
- Seaborn <https://seaborn.pydata.org/>
- Matplotlib, https://www.w3schools.com/python/matplotlib_intro.asp
- Numpy statistics routines
 - <https://docs.scipy.org/doc/numpy/reference/routines.statistics.html>
- SciPy stats
 - <https://docs.scipy.org/doc/scipy-1.15.0/reference/stats.html>



Next steps

Learning data science with Python takes time! Good to see it as a journey, not a destination

- Contact us anytime!
- Many tutorials available to learn, relatively quickly, how to do the basics in Python
 - Data types (integers, float/decimal, strings, lists, arrays)
 - Control flow, programming if-then-else statements, for loops, while loops, conditionals
 - Writing functions
- Data wrangling
 - Loading data from CSV, Excel, databases
 - Handling missing values
 - Filtering data, sorting data, grouping data
- Matplotlib and Seaborn visualizations for Exploratory Data Analysis and final publication graphics



Next steps

Work on a small subset of your data first, or work with a built-in dataset to begin to understand the basics

- Take it a step at a time
- Get to know your data
- Don't try to learn everything at once
- Establish reasonable goals
- Use built-in data and small subsets of larger datasets first
- Follow tutorials, documentation, books and programming resources
 - <https://seaborn.pydata.org/tutorial.html>
 - <https://www.w3schools.com/python/pandas/default.asp>
 - https://pandas.pydata.org/docs/user_guide/10min.html
 - <https://stackoverflow.com/>
 - Software Carpentry <https://swcarpentry.github.io/python-novice-gapminder/>

More at <https://github.com/Dartmouth-Libraries/reproducible-research>



Resources & Links

- Contact us: <https://rc.Dartmouth.edu>
- Upcoming workshops: dartgo.org/rradworkshops
- Materials will be sent out
 - Slides
 - Code and sample datasets to experiment with



Live Code with built-in dataset

Visit <https://dartgo.org/python-data> for external data

```
pip install statsmodels
```

```
import pandas as pd
```

```
from statsmodels.api import datasets
```

```
# Load the mtcars dataset
```

```
mtcars = datasets.get_rdataset("mtcars").data
```

```
print(mtcars.head())
```



Live Code with built in dataset

```
# Check for missing values
```

```
print(mtcars.isnull().sum())
```

```
# For this dataset, we can check for and fill missing values as needed
```

```
# As an example, let's replace missing values in 'hp' with the mean of the column
```

```
mtcars['hp'].fillna(mtcars['hp'].mean(), inplace=True)
```

```
print(mtcars['hp'].describe())
```



Live Code with built in dataset

```
import matplotlib.pyplot as plt

# Create a scatter plot of 'mpg' vs 'wt' (miles per gallon vs weight)

plt.scatter(mtcars['wt'], mtcars['mpg'], color='blue')

plt.xlabel('Weight (1000 lbs)')

plt.ylabel('Miles per Gallon (mpg)')

plt.title('Scatter plot of mpg vs. wt')

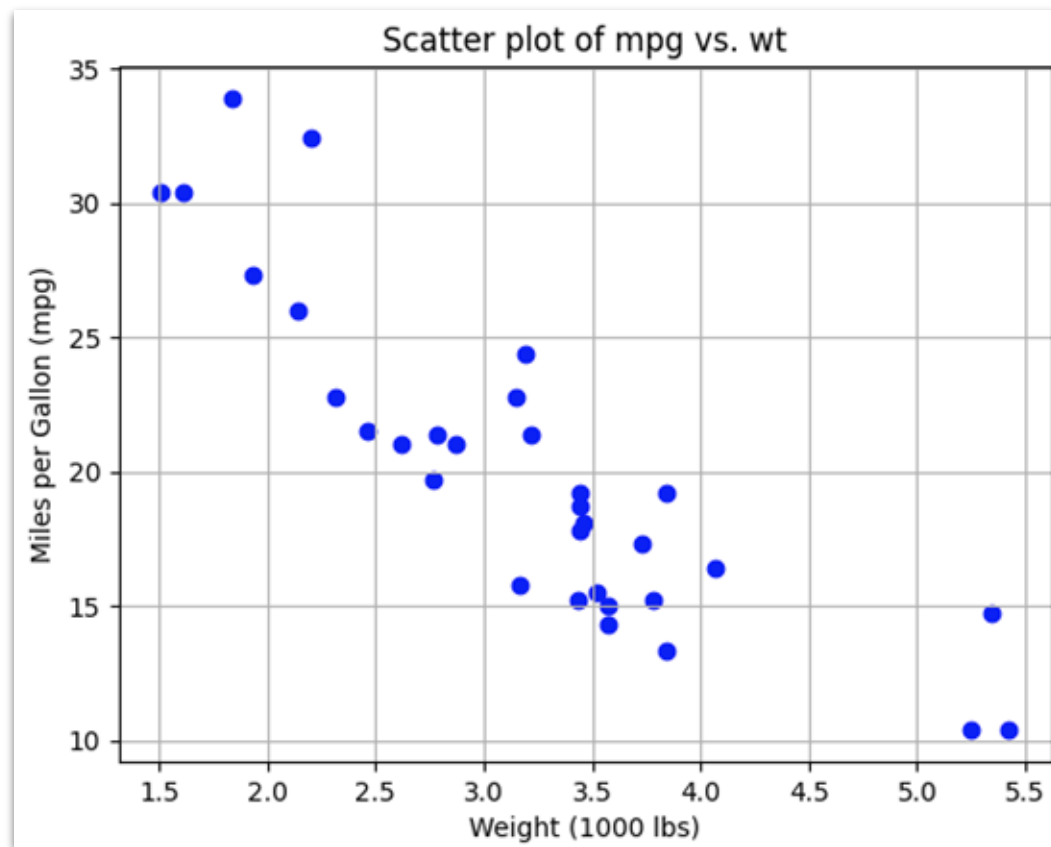
plt.grid()

plt.show()
```



Result

```
import matplotlib.pyplot as plt
# Create a scatter plot of 'mpg' vs
'wt' (miles per gallon vs weight)
plt.scatter(mtcars['wt'],
mtcars['mpg'], color='blue')
plt.xlabel('Weight (1000 lbs)')
plt.ylabel('Miles per Gallon (mpg)')
plt.title('Scatter plot of mpg vs.
wt')
plt.grid()
plt.show()
```

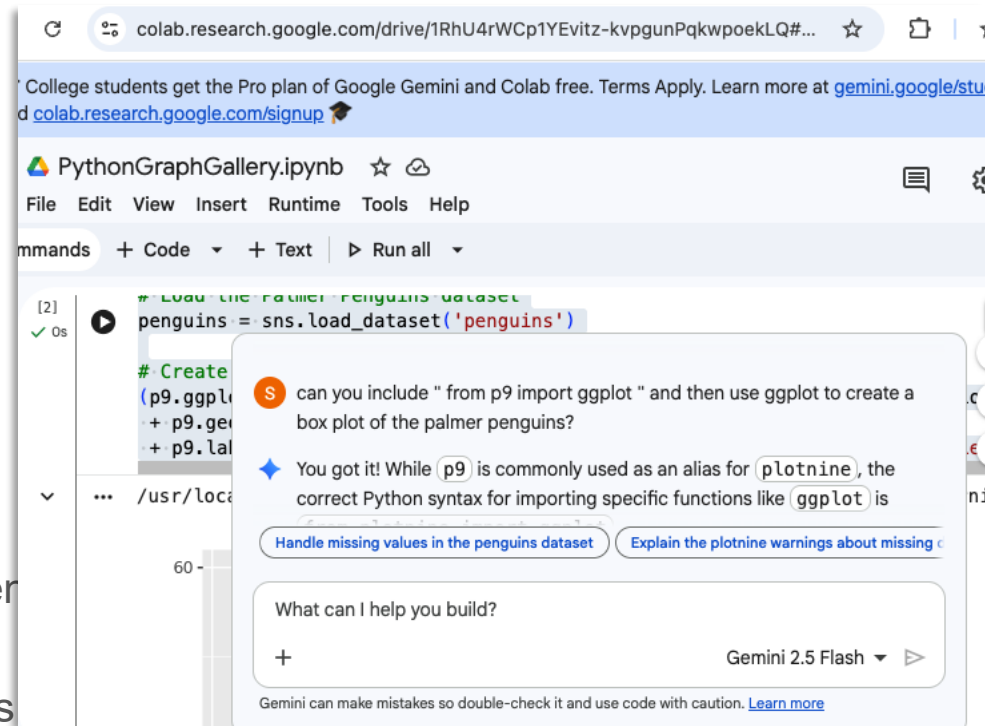




ggplot / R

Know R or GGPlot syntax??

```
import plotnine as p9
import pandas as pd
import seaborn as sns
# Load the Palmer Penguins dataset
penguins = sns.load_dataset('penguins')
# Create a ggplot scatter plot
(p9.ggplot(penguins, p9.aes(x='flipper_length_mm',
                           y='bill_length_mm'))
 + p9.geom_point()
 + p9.labs(title='Flipper Length vs Bill Length (mm)'))
```



Also shown: Google Gemini AI coding



Feedback

Thanks for coming to our workshop!

We want to now learn from you about how we continue to present relevant workshops in the best way we can.

Please take a minute or so to fill out our form with your constructive feedback, we can't wait to hear from you!

dartgo.org/feedback



Questions?

As always, feel free to [reach out anytime](#).

<https://rc.dartmouth.edu/>

<https://services.dartmouth.edu/>

Thanks for attending our workshop!

